Statistics Lab Programs using Python

Sampling, Hypothesis Testing (Z, t, F, ANOVA) and Chi-square Tests

No.	Program	Topic
1	Exam marks of a university	Population, Sample and Random Sampling
2	Courier delivery times	Sampling Distribution, Standard Error of the Mean
3	Tea packet filling machine	Z-Test: Procedure, Decision Rule, Interpretation
4	LED bulb life claim	One-tailed vs Two-tailed Tests
5	Coffee shop waiting time	One-Sample t-Test
6	Two teaching methods	Two Independent Samples t-Test
7	Blood pressure before & after yoga	Two Related (Paired) Samples t-Test
8	Bolt diameters from two machines	F-Test for Equality of Variances
9	Crop yield with three fertilizers	One-way ANOVA
10	Fertilizer × Irrigation crop yield	Two-way ANOVA
11	Is the die fair?	Chi-square Goodness of Fit
12	Gender vs learning mode preference	Chi-square Test of Independence

Libraries used: numpy, scipy, matplotlib, pandas, statsmodels (only Program 10).

Install if needed: pip install numpy scipy matplotlib pandas statsmodels

General steps of any hypothesis test

1. State H_0 (null) and H_1 (alternative).
2. Choose significance level α (usually 0.05).
3. Compute the test statistic (Z, t, F or χ^2).
4. Find the p-value / critical value.
5. **Decision rule:** if p-value < α (or |statistic| > critical value) → Reject H_0; otherwise → Fail to reject H_0.
6. Write the conclusion in the words of the problem.

Program 1: Populations, Samples and Random Sampling

Aim: To understand the difference between a population and a sample, and to draw a simple random sample using Python.

Scenario: A university has **10,000 students**. Their exam marks (out of 100) form the *population*. The examination controller cannot check all records, so she selects **50 students at random** (a *sample*) to estimate the average mark.

Key terms

- **Population (N):** the complete group we want to study → parameters μ (mean), σ (std dev).
- **Sample (n):** a subset of the population → statistics $\bar{x}$ (mean), s (std dev).
- **Simple random sampling:** every member has an equal chance of selection.
- **Sampling error:** $\bar{x} - \mu$ (the difference caused only by taking a sample).

Syntax

Syntax	Parameters	Why it is used in this program
np.random.seed(seed)	seed – any whole number (here 42)	Fixes the random numbers so the same output comes every time the program is run.
np.random.normal(loc, scale, size)	loc – mean of the distribution (65) scale – standard deviation (12) size – how many values to create (10000)	Creates the population: marks of 10,000 students.
np.clip(a, a_min, a_max)	a – array of values a_min – lowest allowed value (0) a_max – highest allowed value (100)	Keeps every mark between 0 and 100.
np.random.choice(a, size, replace)	a – array to pick from (population) size – number of items to pick (50) replace=False – an item cannot be picked twice	Draws a simple random sample of 50 students.
array.mean() / array.std(ddof)	ddof=0 (default) – divides by N: population SD σ ddof=1 – divides by n−1: sample SD s	Finds μ and σ of the population and $\bar{x}$ and s of the sample.
plt.subplots(nrows, ncols, figsize)	nrows, ncols – number of graphs in rows and columns (1, 2) figsize – (width, height) of the figure in inches	Places the population and sample histograms side by side.

Syntax	Parameters	Why it is used in this program
ax.hist(x, bins, color, edgecolor)	x – data to plot bins – number of bars color / edgecolor – fill and border colour	Draws the histograms of marks.
ax.axvline(x, color, linestyle, label)	x – position of the vertical line linestyle – '--' dashed, ':' dotted label – text shown in the legend	Marks the population mean and sample mean on the graph.

Program Code

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)          # to get the same result every time

# Step 1: Create the population - marks of 10,000 students
population = np.random.normal(loc=65, scale=12, size=10000).round()
population = np.clip(population, 0, 100)      # marks must be between 0 and 100

# Step 2: Draw a simple random sample of 50 students (without replacement)
sample = np.random.choice(population, size=50, replace=False)

# Step 3: Compare population parameters and sample statistics
print("POPULATION  (N = 10000)")
print(f"  Population mean  (mu)    = {population.mean():.2f}")
print(f"  Population SD    (sigma) = {population.std():.2f}")
print("SAMPLE      (n = 50)")
print(f"  Sample mean      (x-bar) = {sample.mean():.2f}")
print(f"  Sample SD        (s)     = {sample.std(ddof=1):.2f}")
print(f"Sampling error (x-bar - mu) = {sample.mean() - population.mean():.2f}")

# Step 4: Different random samples give different sample means
print("\nMeans of 5 different random samples of size 50:")
for i in range(1, 6):
    s = np.random.choice(population, size=50, replace=False)
    print(f"  Sample {i}: mean = {s.mean():.2f}")

# Step 5: Graph
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].hist(population, bins=30, color='skyblue', edgecolor='black')
ax[0].axvline(population.mean(), color='red', linestyle='--', label=f'Population mean =
{population.mean():.1f}')
ax[0].set_title('Population: Marks of 10,000 Students')
ax[0].set_xlabel('Marks'); ax[0].set_ylabel('Number of students'); ax[0].legend()

ax[1].hist(sample, bins=10, color='orange', edgecolor='black')
```

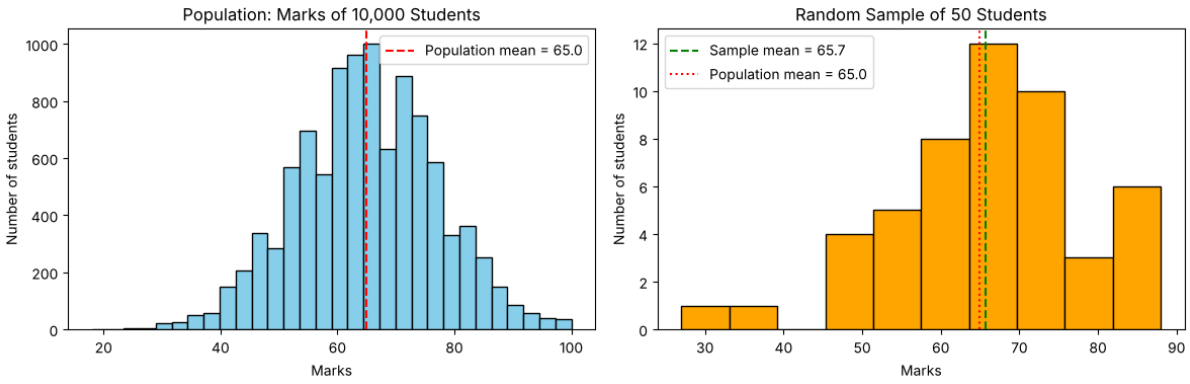
```
ax[1].axvline(sample.mean(), color='green', linestyle='--', label=f'Sample mean =
{sample.mean():.1f}')
ax[1].axvline(population.mean(), color='red', linestyle=':', label=f'Population mean =
{population.mean():.1f}')
ax[1].set_title('Random Sample of 50 Students')
ax[1].set_xlabel('Marks'); ax[1].set_ylabel('Number of students'); ax[1].legend()
plt.tight_layout()
plt.show()
```

Output

```
POPULATION  (N = 10000)
  Population mean  (mu)    = 64.96
  Population SD    (sigma) = 12.02
SAMPLE      (n = 50)
  Sample mean      (x-bar) = 65.70
  Sample SD        (s)     = 12.67
Sampling error (x-bar - mu) = 0.74

Means of 5 different random samples of size 50:
  Sample 1: mean = 64.14
  Sample 2: mean = 65.54
  Sample 3: mean = 64.92
  Sample 4: mean = 65.02
  Sample 5: mean = 64.74
```

Graph



Result

The population mean of 10,000 students is 64.96 and the mean of a random sample of 50 students is 65.70, a sampling error of only 0.74. Each new random sample gives a slightly different mean (64.14 to 65.54), all close to the population mean. A properly drawn random sample represents the population well.

Program 2: Sampling Distribution and Standard Error of the Mean

Aim: To build the sampling distribution of the sample mean and compute the Standard Error (SE) of the mean for different sample sizes.

Scenario: A courier company in Chennai records the **delivery time (minutes)** of 1,00,000 parcels (population mean about 40 minutes, SD about 8 minutes). The manager takes **1000 random samples** each of size **n = 10, 30 and 100**, finds the mean of every sample, and studies how these sample means are spread.

Key terms and formulas

- **Sampling distribution of the mean:** the distribution of the means of many samples of the same size.
- The mean of all sample means ≈ population mean μ.
- **Standard Error of the Mean (SE):** the standard deviation of the sample means. SE = σ / √n
- When σ is not known, SE is estimated from one sample as **s / √n**.
- Larger sample size → smaller SE → sample mean is closer to μ.

Syntax

Syntax	Parameters	Why it is used in this program
np.random.normal(loc, scale, size)	loc – mean (40 minutes) scale – SD (8 minutes) size – 100000 parcels	Creates the population of delivery times.
np.random.choice(a, size, replace)	a – population size – sample size n (10, 30 or 100) replace=False – no parcel chosen twice	Draws one random sample of n parcels.
[expression for _ in range(k)]	expression – what to compute each time (mean of one sample) k – how many times to repeat (1000)	Takes 1000 samples and stores the 1000 sample means in a list.
np.sqrt(x)	x – number (sample size n)	Used in the formula SE = σ / √n.
np.std(a)	a – list of the 1000 sample means	Finds the actual spread of the sample means, to compare with σ/√n.
ax.set_xlim(left, right)	left, right – smallest and largest x value shown (30, 50)	Uses the same scale on all three graphs so their widths can be compared.

Program Code

```
import numpy as np
```

```
import matplotlib.pyplot as plt

np.random.seed(1)

# Step 1: Population of delivery times (minutes)
population = np.random.normal(loc=40, scale=8, size=100000)
mu, sigma = population.mean(), population.std()
print(f"Population mean = {mu:.2f} min, Population SD = {sigma:.2f} min\n")

# Step 2: Take 1000 samples for each sample size and store their means
sample_sizes = [10, 30, 100]
num_samples = 1000

fig, ax = plt.subplots(2, 2, figsize=(12, 8))
ax = ax.ravel()
ax[0].hist(population, bins=50, color='lightgray', edgecolor='black')
ax[0].axvline(mu, color='red', linestyle='--', label=f'Population mean = {mu:.1f}')
ax[0].set_title('Population: Delivery Times of 1,00,000 Parcels')
ax[0].set_xlabel('Minutes'); ax[0].set_ylabel('Frequency'); ax[0].legend()

print(f"{'n':>5} {'Mean of sample means':>22} {'SE = σ/√n':>12} {'SD of sample
means':>20}")
for i, n in enumerate(sample_sizes):
    sample_means = [np.random.choice(population, size=n, replace=False).mean() for _ in
range(num_samples)]
    se_formula = sigma / np.sqrt(n)
    se_observed = np.std(sample_means)
    print(f"{n:>5} {np.mean(sample_means):>22.2f} {se_formula:>12.2f}
{se_observed:>20.2f}")

    ax[i+1].hist(sample_means, bins=30, color='lightgreen', edgecolor='black')
    ax[i+1].axvline(mu, color='red', linestyle='--', label='Population mean')
    ax[i+1].set_xlim(30, 50)          # same scale so the spread can be compared
    ax[i+1].set_title(f'Sampling Distribution of Mean (n = {n}), SE = {se_formula:.2f}')
    ax[i+1].set_xlabel('Sample mean (minutes)'); ax[i+1].set_ylabel('Frequency');
ax[i+1].legend()

plt.tight_layout()
plt.show()

# Step 3: SE estimated from ONE sample of 30 parcels (sigma not known)
one_sample = np.random.choice(population, size=30, replace=False)
se_est = one_sample.std(ddof=1) / np.sqrt(30)
print(f"\nOne sample of 30 parcels: mean = {one_sample.mean():.2f}, estimated SE = s/√n
= {se_est:.2f}")
```

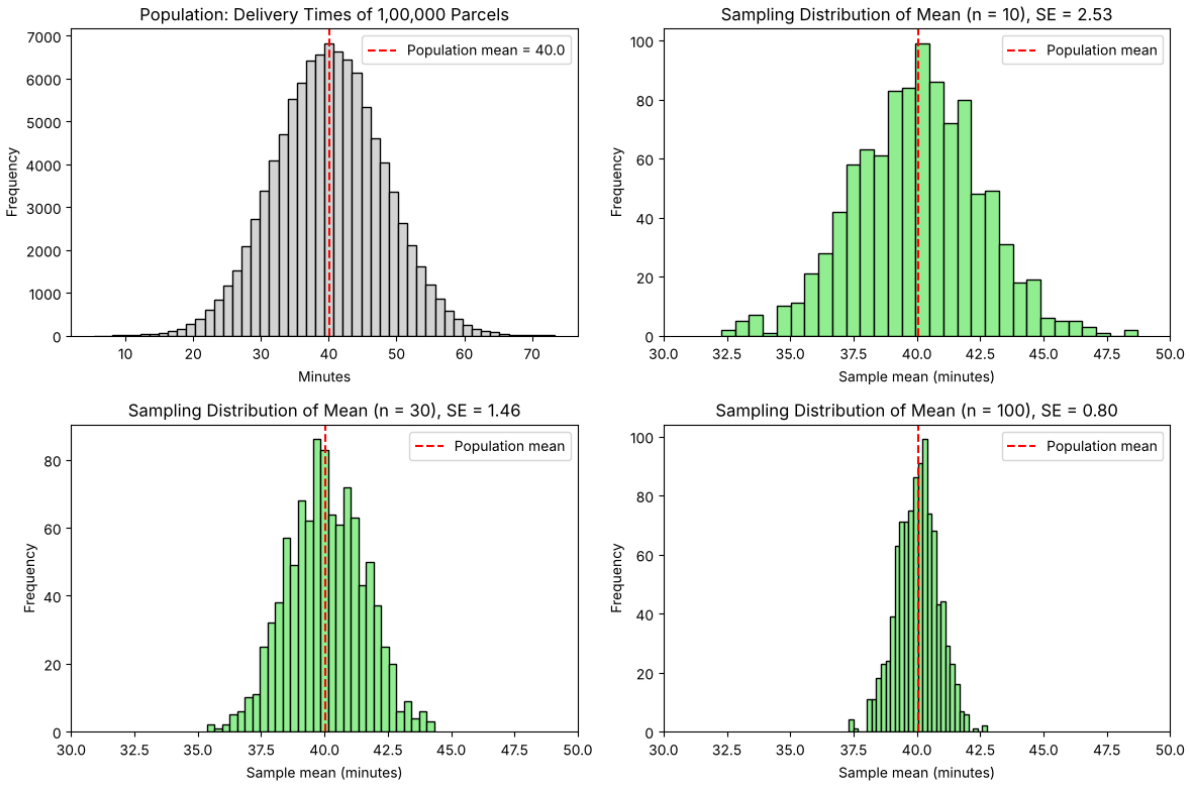
Output

```
Population mean = 40.04 min, Population SD = 7.99 min
```

n	Mean of sample means	SE = σ/√n	SD of sample means
10	40.03	2.53	2.49
30	40.06	1.46	1.52
100	40.01	0.80	0.81

One sample of 30 parcels: mean = 39.50, estimated SE = s/√n = 1.28

Graph



Result

The mean of the 1000 sample means stays at about 40 minutes (equal to the population mean μ = 40.04) for every sample size. The standard error falls as the sample size grows: 2.53 for n = 10, 1.46 for n = 30 and 0.80 for n = 100, and these values match the actual spread of the sample means (2.49, 1.52, 0.81). The graphs show the sampling distribution becoming narrower as n increases, so larger samples give more precise estimates of the population mean.

Program 3: Z-Test – Procedure, Decision Rule and Interpretation

Aim: To test whether a tea-packet filling machine is working correctly using a one-sample Z-test.

Scenario: A tea company's machine is set to fill **500 g** per packet. From past records the population SD is known: **σ = 8 g**. A quality inspector weighs **36 randomly chosen packets**. Is the machine filling correctly? (α = 0.05)

Procedure

1. H_0: μ = 500 g (machine is working correctly)
2. H_1: μ ≠ 500 g (machine is not working correctly) → **two-tailed**
3. α = 0.05 → critical values Z = ±1.96
4. Test statistic: $\mathbf{Z = (\bar{x} - \mu_0) / (\sigma / \sqrt{n})}$
5. **Decision rule:** Reject H_0 if |Z| > 1.96 (or p-value < 0.05)

Z-test is used when σ is known (or n ≥ 30).

Syntax

Syntax	Parameters	Why it is used in this program
(x_bar - mu0) / (sigma / np.sqrt(n))	x_bar – sample mean mu0 – claimed mean under H_0 (500) sigma – known population SD (8) n – sample size (36)	Calculates the Z statistic.
stats.norm.ppf(q)	q – area to the left (1 − α/2 = 0.975)	Gives the critical value 1.96 for a two-tailed test at α = 0.05.
stats.norm.cdf(x)	x – a Z value	Gives the area to the left of Z; used to find the p-value = 2 × (1 − cdf of the absolute value of Z).
stats.norm.pdf(x)	x – array of Z values from −4 to 4	Gives the height of the normal curve so it can be drawn.
ax.fill_between(x, y, where, color, alpha)	x, y – points of the curve where – condition that selects the region to shade alpha – transparency (0 to 1)	Shades the rejection regions red and the acceptance region green.

Program Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

```
# Step 1: Data - weights (g) of 36 packets
weights = np.array([495, 498, 492, 501, 497, 494, 499, 496, 503, 490, 497, 500,
                    493, 498, 495, 502, 491, 497, 499, 494, 496, 500, 492, 498,
                    497, 495, 501, 493, 496, 499, 494, 498, 502, 495, 497, 496])
mu0 = 500        # claimed mean
sigma = 8        # known population SD
alpha = 0.05
n = len(weights)
x_bar = weights.mean()

# Step 2: Z statistic
se = sigma / np.sqrt(n)
z = (x_bar - mu0) / se

# Step 3: Critical value and p-value (two-tailed)
z_critical = stats.norm.ppf(1 - alpha/2)
p_value = 2 * (1 - stats.norm.cdf(abs(z)))

print(f"Sample size (n)      = {n}")
print(f"Sample mean (x-bar)  = {x_bar:.2f} g")
print(f"Standard error       = {se:.4f}")
print(f"Z statistic          = {z:.4f}")
print(f"Critical value       = ±{z_critical:.2f}")
print(f"p-value              = {p_value:.4f}")

# Step 4: Decision rule
if abs(z) > z_critical:
    print("\nDecision: |Z| > 1.96 -> Reject H0")
    print("Interpretation: The machine is NOT filling 500 g on average. It needs
adjustment.")
else:
    print("\nDecision: |Z| <= 1.96 -> Fail to reject H0")
    print("Interpretation: The machine is filling 500 g on average.")

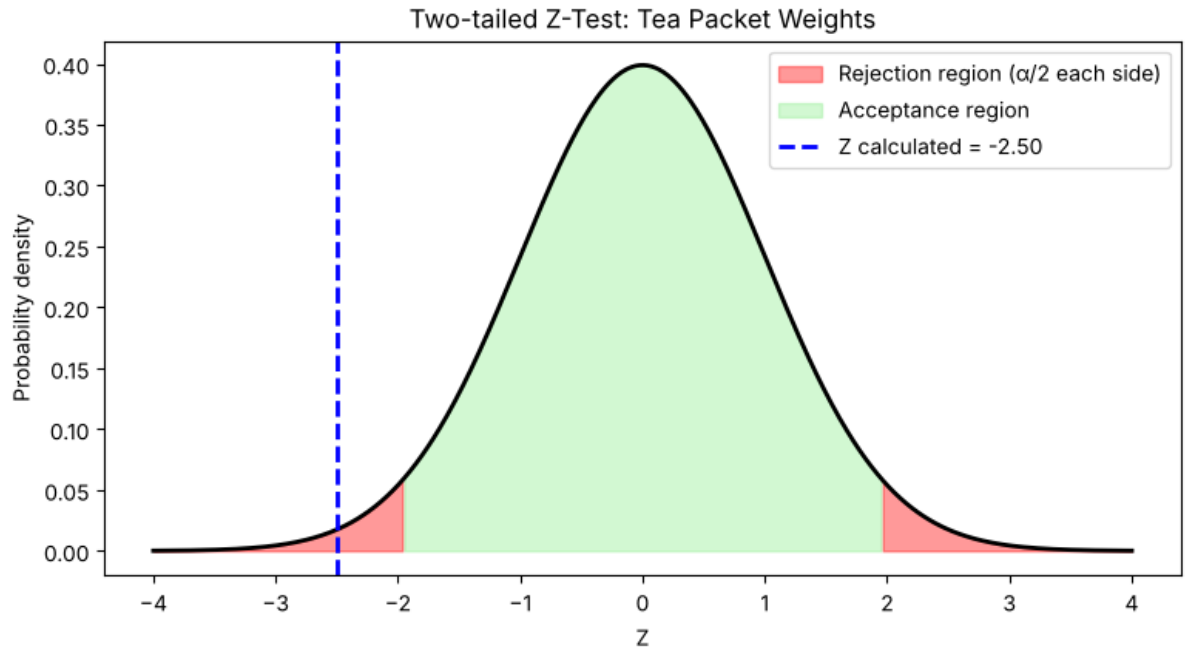
# Step 5: Graph - standard normal curve with rejection regions
x = np.linspace(-4, 4, 500)
y = stats.norm.pdf(x)
plt.figure(figsize=(9, 4.5))
plt.plot(x, y, 'k', lw=2)
plt.fill_between(x, y, where=(x <= -z_critical), color='red', alpha=0.4, label='Rejection
region (α/2 each side)')
plt.fill_between(x, y, where=(x >= z_critical), color='red', alpha=0.4)
plt.fill_between(x, y, where=(abs(x) < z_critical), color='lightgreen', alpha=0.4,
label='Acceptance region')
plt.axvline(z, color='blue', linestyle='--', lw=2, label=f'Z calculated = {z:.2f}')
plt.title('Two-tailed Z-Test: Tea Packet Weights')
plt.xlabel('Z'); plt.ylabel('Probability density'); plt.legend()
plt.show()
```

Output

Sample size (n) = 36
Sample mean (x-bar) = 496.67 g
Standard error = 1.3333
Z statistic = -2.5000
Critical value = ±1.96
p-value = 0.0124

Decision: |Z| > 1.96 -> Reject H0
Interpretation: The machine is NOT filling 500 g on average. It needs adjustment.

Graph



Result

Z = −2.50 and p-value = 0.0124. Since |Z| = 2.50 > 1.96 ($p < 0.05$), H_0 is rejected. The machine is under-filling packets (mean 496.67 g instead of 500 g) and needs to be re-calibrated.

Program 4: One-tailed and Two-tailed Tests

Aim: To understand the difference between one-tailed and two-tailed tests using the same data.

Scenario: An LED bulb company claims its bulbs last **1000 hours** on average ($\sigma = 100$ hours). A consumer forum suspects the bulbs last **less** than claimed. They test **50 bulbs** and find an average life of **975 hours**. ($\alpha = 0.05$)

Test	H_0	H_1	Critical value	Reject H_0 if
Two-tailed	$\mu = 1000$	$\mu \neq 1000$	± 1.96	$\|Z\| > 1.96$
Left-tailed	$\mu \geq 1000$	$\mu < 1000$	-1.645	$Z < -1.645$
Right-tailed	$\mu \leq 1000$	$\mu > 1000$	$+1.645$	$Z > 1.645$

When to use: a one-tailed test is used when the question has a direction ("less than", "more than", "improved"). A two-tailed test is used when we only ask "is it different?".

Syntax

Syntax	Parameters	Why it is used in this program
stats.norm.ppf(1 - alpha/2)	alpha – significance level (0.05)	Two-tailed critical value ±1.96 (α split into both tails).
stats.norm.ppf(alpha)	alpha – significance level (0.05)	Left-tailed critical value −1.645 (whole α in the left tail).
stats.norm.cdf(z)	z – calculated Z statistic	Left-tailed p-value (area to the left of Z).
2 * (1 - stats.norm.cdf(abs(z)))	abs(z) – size of Z without its sign	Two-tailed p-value (area in both tails).
1 - stats.norm.cdf(z)	z – calculated Z statistic	Right-tailed p-value (area to the right of Z).

Program Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Step 1: Given summary data
mu0 = 1000      # claimed mean life (hours)
sigma = 100     # population SD
n = 50
x_bar = 975
alpha = 0.05
```

```
# Step 2: Z statistic (same for both tests)
z = (x_bar - mu0) / (sigma / np.sqrt(n))
print(f"Z statistic = {z:.4f}\n")

# Step 3: Two-tailed test
z_crit_two = stats.norm.ppf(1 - alpha/2)
p_two = 2 * (1 - stats.norm.cdf(abs(z)))
print("TWO-TAILED TEST  (H1: mu != 1000)")
print(f"  Critical values = ±{z_crit_two:.3f},  p-value = {p_two:.4f}")
print("  Decision:", "Reject H0" if p_two < alpha else "Fail to reject H0")

# Step 4: Left-tailed test
z_crit_left = stats.norm.ppf(alpha)
p_left = stats.norm.cdf(z)
print("\nLEFT-TAILED TEST (H1: mu < 1000)")
print(f"  Critical value = {z_crit_left:.3f},  p-value = {p_left:.4f}")
print("  Decision:", "Reject H0" if p_left < alpha else "Fail to reject H0")

# Step 5: Right-tailed (for comparison only)
p_right = 1 - stats.norm.cdf(z)
print("\nRIGHT-TAILED TEST (H1: mu > 1000)")
print(f"  Critical value = {stats.norm.ppf(1-alpha):.3f},  p-value = {p_right:.4f}")
print("  Decision:", "Reject H0" if p_right < alpha else "Fail to reject H0")

# Step 6: Graph - two-tailed vs left-tailed rejection regions
x = np.linspace(-4, 4, 500)
y = stats.norm.pdf(x)
fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))

ax[0].plot(x, y, 'k', lw=2)
ax[0].fill_between(x, y, where=(x <= -z_crit_two), color='red', alpha=0.4, label='Rejection
region (2.5% each)')
ax[0].fill_between(x, y, where=(x >= z_crit_two), color='red', alpha=0.4)
ax[0].axvline(z, color='blue', linestyle='--', lw=2, label=f'Z = {z:.2f}')
ax[0].set_title(f'Two-tailed Test (p = {p_two:.3f})')
ax[0].set_xlabel('Z'); ax[0].legend()

ax[1].plot(x, y, 'k', lw=2)
ax[1].fill_between(x, y, where=(x <= z_crit_left), color='red', alpha=0.4, label='Rejection
region (5% left)')
ax[1].axvline(z, color='blue', linestyle='--', lw=2, label=f'Z = {z:.2f}')
ax[1].set_title(f'Left-tailed Test (p = {p_left:.3f})')
ax[1].set_xlabel('Z'); ax[1].legend()
plt.tight_layout()
plt.show()
```

Output

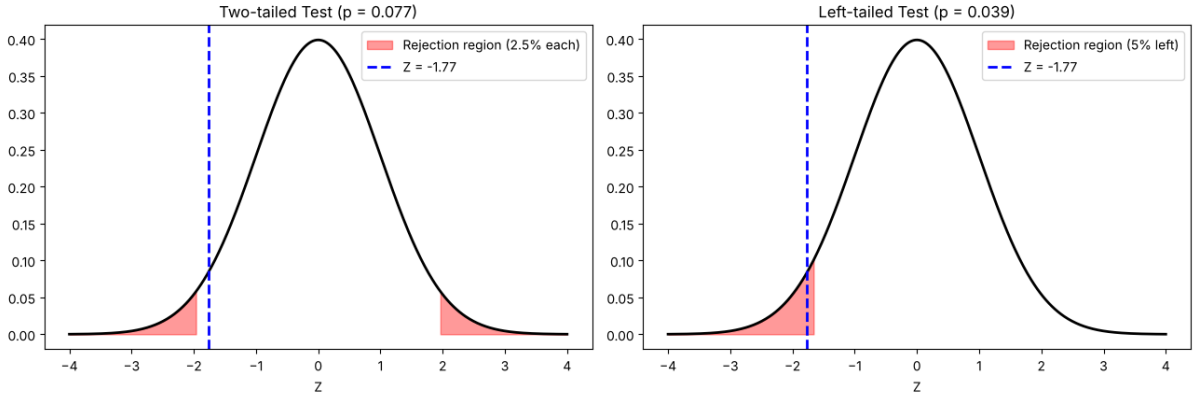
```
Z statistic = -1.7678
```

TWO-TAILED TEST (H1: mu != 1000)
  Critical values = ±1.960, p-value = 0.0771
  Decision: Fail to reject H0

LEFT-TAILED TEST (H1: mu < 1000)
  Critical value = -1.645, p-value = 0.0385
  Decision: Reject H0

RIGHT-TAILED TEST (H1: mu > 1000)
  Critical value = 1.645, p-value = 0.9615
  Decision: Fail to reject H0

Graph



Result

Z = −1.77 for both tests. In the **two-tailed** test p = 0.077 > 0.05, so H_0 is not rejected. In the **left-tailed** test p = 0.039 < 0.05, so H_0 is rejected: the bulbs last less than 1000 hours. The same data gives different conclusions because a one-tailed test puts the whole 5% rejection area on one side (critical value −1.645 instead of −1.96). The direction of H_1 must therefore be decided *before* looking at the data.

Program 5: One-Sample t-Test

Aim: To test whether a sample mean differs from a claimed value when σ is unknown and the sample is small.

Scenario: A coffee shop advertises that customers wait **5 minutes** on average for their order. A student records the waiting time of **15 customers**. Is the actual waiting time different from 5 minutes? (α = 0.05)

- H_0: μ = 5 minutes H_1: μ ≠ 5 minutes (two-tailed)
- Test statistic: $\mathbf{t = (\bar{x} - \mu_0) / (s / \sqrt{n})}$, degrees of freedom = n − 1
- *t-test is used when σ is unknown and n < 30.*

Syntax

Syntax	Parameters	Why it is used in this program
stats.ttest_1samp(a, popmean)	a – sample data (15 waiting times) popmean – claimed mean under H_0 (5) Returns – t statistic and two-tailed p-value	Performs the one-sample t-test.
stats.t.ppf(q, df)	q – area to the left (0.975) df – degrees of freedom (n − 1 = 14)	Gives the two-tailed t critical value 2.145.
stats.t.pdf(x, df)	x – array of t values df – degrees of freedom	Gives the height of the t-curve so it can be drawn.
ax.bar(x, height, color, edgecolor)	x – position of each bar (customer number) height – waiting time of each customer	Shows each customer's waiting time.
ax.axhline(y, color, linestyle, label)	y – position of the horizontal line	Marks the claimed mean (5) and the sample mean on the bar chart.

Program Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Step 1: Data - waiting times (minutes) of 15 customers
wait = np.array([5.8, 6.2, 5.1, 4.9, 6.5, 5.7, 6.0, 5.4, 6.8, 5.3, 5.9, 6.1, 4.8, 5.6, 6.3])
mu0 = 5
alpha = 0.05
n = len(wait)
```

```
df = n - 1

# Step 2: t-test using scipy
t_stat, p_value = stats.ttest_1samp(wait, popmean=mu0)
t_critical = stats.t.ppf(1 - alpha/2, df)

print(f"Sample mean = {wait.mean():.3f} min, Sample SD = {wait.std(ddof=1):.3f}")
print(f"Degrees of freedom = {df}")
print(f"t statistic  = {t_stat:.4f}")
print(f"t critical   = ±{t_critical:.4f}")
print(f"p-value      = {p_value:.6f}")

# Step 3: Decision
if p_value < alpha:
    print("\nDecision: Reject H0 -> Average waiting time is significantly different from 5
minutes.")
else:
    print("\nDecision: Fail to reject H0 -> No significant difference from 5 minutes.")

# Step 4: Graphs
fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
ax[0].bar(range(1, n+1), wait, color='peru', edgecolor='black')
ax[0].axhline(mu0, color='red', linestyle='--', lw=2, label='Claimed mean = 5 min')
ax[0].axhline(wait.mean(), color='blue', linestyle='-', lw=2, label=f'Sample mean =
{wait.mean():.2f} min')
ax[0].set_title('Waiting Time of 15 Customers')
ax[0].set_xlabel('Customer'); ax[0].set_ylabel('Minutes'); ax[0].legend()

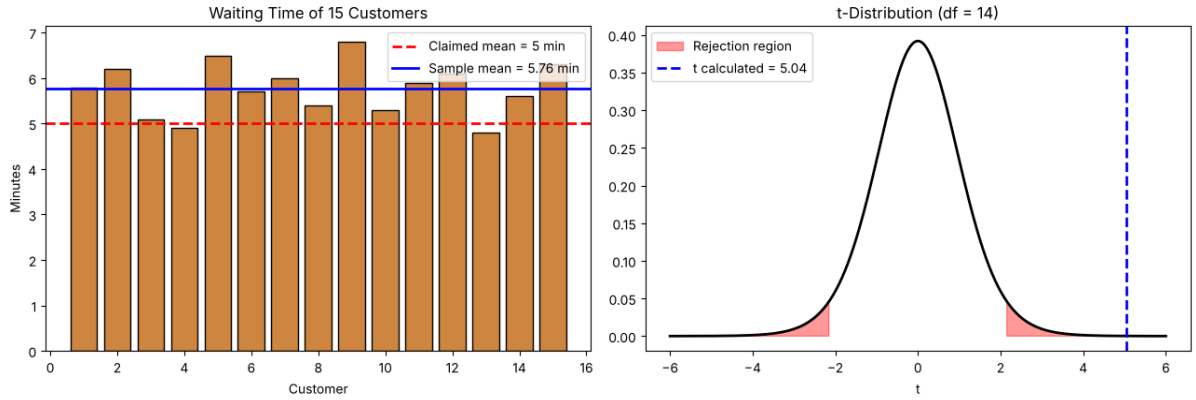
x = np.linspace(-6, 6, 500)
y = stats.t.pdf(x, df)
ax[1].plot(x, y, 'k', lw=2)
ax[1].fill_between(x, y, where=(abs(x) >= t_critical), color='red', alpha=0.4,
label='Rejection region')
ax[1].axvline(t_stat, color='blue', linestyle='--', lw=2, label=f't calculated = {t_stat:.2f}')
ax[1].set_title(f't-Distribution (df = {df})')
ax[1].set_xlabel('t'); ax[1].legend()
plt.tight_layout()
plt.show()
```

Output

```
Sample mean = 5.760 min, Sample SD = 0.584
Degrees of freedom = 14
t statistic  = 5.0395
t critical   = ±2.1448
p-value      = 0.000181

Decision: Reject H0 -> Average waiting time is significantly different from 5 minutes.
```

Graph



Result

t = 5.04 (df = 14), p = 0.00018. Since |t| > 2.145 (p < 0.05), H_0 is rejected. Customers actually wait about 5.76 minutes on average, which is significantly more than the advertised 5 minutes.

Program 6: t-Test for Two Independent Samples

Aim: To compare the means of two independent groups.

Scenario: A school wants to know whether **activity-based learning (Method B)** gives different marks from the **traditional lecture method (Method A)**. Two different sections of 12 students each were taught by the two methods, and their test marks were recorded. ($\alpha = 0.05$)

- H_0: $\mu_A = \mu_B$ (both methods give the same average marks)
- H_1: $\mu_A \neq \mu_B$
- Test statistic: $t = (\bar{x}_1 - \bar{x}_2) / \sqrt{(s^2_p(1/n_1 + 1/n_2))}$, $df = n_1 + n_2 - 2$
- *Independent* = different people in each group.

Syntax

Syntax	Parameters	Why it is used in this program
stats.ttest_ind(a, b, equal_var)	a – marks of Method A b – marks of Method B equal_var=True – assumes equal variances (pooled t-test) Returns – t statistic and two-tailed p-value	Compares the means of two different groups of students.
stats.t.ppf(q, df)	q – 0.975 for two-tailed $\alpha = 0.05$ df – $n_1 + n_2 - 2 = 22$	Gives the t critical value 2.074.
ax.boxplot(x, patch_artist, boxprops)	x – list of the groups to plot patch_artist=True – allows the boxes to be filled with colour boxprops – box style, e.g. dict(facecolor='lightblue')	Shows the spread of marks in each group.
ax.set_xticks(ticks) / ax.set_xticklabels(labels)	ticks – positions [1, 2] labels – names shown below each box	Names the boxes 'Method A' and 'Method B'.
ax.bar(x, height, yerr, capsize)	height – group means yerr – error bar size (SD of each group) capsize – width of the error bar caps	Compares the mean marks with their spread.

Program Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Step 1: Data - marks of two different groups of students
method_A = np.array([65, 70, 68, 72, 60, 66, 69, 71, 64, 67, 63, 70])   # Lecture
method_B = np.array([72, 78, 75, 80, 74, 77, 79, 73, 76, 81, 70, 75])   # Activity-based
```

```
alpha = 0.05

# Step 2: Independent two-sample t-test (equal variances assumed)
t_stat, p_value = stats.ttest_ind(method_A, method_B, equal_var=True)
df = len(method_A) + len(method_B) - 2
t_critical = stats.t.ppf(1 - alpha/2, df)

print(f"Method A: mean = {method_A.mean():.2f}, SD = {method_A.std(ddof=1):.2f}")
print(f"Method B: mean = {method_B.mean():.2f}, SD = {method_B.std(ddof=1):.2f}")
print(f"Degrees of freedom = {df}")
print(f"t statistic = {t_stat:.4f}")
print(f"t critical  = ±{t_critical:.4f}")
print(f"p-value     = {p_value:.6f}")

# Step 3: Decision
if p_value < alpha:
    print("\nDecision: Reject H0 -> The two teaching methods give significantly different
marks.")
else:
    print("\nDecision: Fail to reject H0 -> No significant difference between the methods.")

# Step 4: Graphs - boxplot and mean bar chart
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
ax[0].boxplot([method_A, method_B], patch_artist=True,
boxprops=dict(facecolor='lightblue'))
ax[0].set_xticks([1, 2]); ax[0].set_xticklabels(['Method A\n(Lecture)', 'Method
B\n(Activity)'])
ax[0].set_title('Distribution of Marks'); ax[0].set_ylabel('Marks')

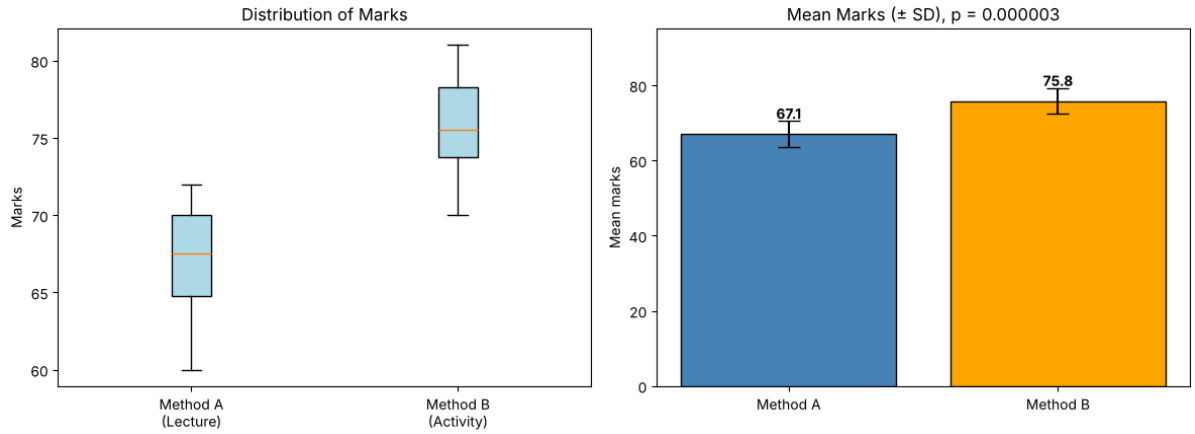
means = [method_A.mean(), method_B.mean()]
errors = [method_A.std(ddof=1), method_B.std(ddof=1)]
ax[1].bar(['Method A', 'Method B'], means, yerr=errors, capsize=8, color=['steelblue',
'orange'], edgecolor='black')
for i, m in enumerate(means):
    ax[1].text(i, m + 4, f'{m:.1f}', ha='center', fontweight='bold')
ax[1].set_title(f'Mean Marks (± SD), p = {p_value:.6f}'); ax[1].set_ylabel('Mean marks')
ax[1].set_ylim(0, 95)
plt.tight_layout()
plt.show()
```

Output

```
Method A: mean = 67.08, SD = 3.60
Method B: mean = 75.83, SD = 3.33
Degrees of freedom = 22
t statistic = -6.1804
t critical  = ±2.0739
p-value     = 0.000003

Decision: Reject H0 -> The two teaching methods give significantly different marks.
```

Graph



Result

t = −6.18 (df = 22), p = 0.000003. Since |t| > 2.074, H_0 is rejected. Activity-based learning (mean 75.83) produces significantly higher marks than the lecture method (mean 67.08).

Program 7: t-Test for Two Related (Paired) Samples

Aim: To test whether there is a significant change in measurements taken on the same subjects before and after a treatment.

Scenario: A hospital measures the **systolic blood pressure (mmHg)** of **10 patients** before and after an **8-week yoga programme**. Did yoga reduce blood pressure? ($\alpha = 0.05$)

- H_0: $\mu_d = 0$ (no change in BP), where d = Before − After
- H_1: $\mu_d > 0$ (BP decreased after yoga) → **one-tailed (right)**
- Test statistic: $t = \bar{d} / (s_d / \sqrt{n})$, df = n − 1
- *Related / paired* = same people measured twice.

Syntax

Syntax	Parameters	Why it is used in this program
stats.ttest_rel(a, b, alternative)	a – BP before yoga b – BP after yoga (same patients, same order) alternative – 'two-sided' (default), 'greater' (a > b) or 'less' (a < b) Returns – t statistic and p-value	Paired t-test; alternative='greater' tests whether BP before is higher than after (BP reduced).
before - after	Subtracts element by element	Finds the reduction d for each patient.
stats.t.ppf(1 - alpha, df)	1 − alpha – 0.95 (whole α in one tail) df – n − 1 = 9	Gives the one-tailed t critical value 1.833.
ax.plot(x, y, marker, color)	x – ['Before', 'After'] y – the two BP values of one patient marker='o' – draws a dot at each point	Draws one line per patient to show the change.

Program Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Step 1: Data - BP of the same 10 patients
before = np.array([150, 145, 160, 155, 148, 152, 158, 149, 162, 147])
after  = np.array([142, 140, 151, 150, 145, 146, 150, 147, 153, 141])
diff = before - after
alpha = 0.05
n = len(diff)

# Step 2: Paired t-test (one-tailed: before > after)
```

```
t_stat, p_value = stats.ttest_rel(before, after, alternative='greater')
t_critical = stats.t.ppf(1 - alpha, n - 1)

print("Differences (Before - After):", diff)
print(f"Mean difference = {diff.mean():.2f} mmHg, SD of differences =
{diff.std(ddof=1):.2f}")
print(f"t statistic = {t_stat:.4f}")
print(f"t critical  = {t_critical:.4f} (one-tailed, df = {n-1})")
print(f"p-value     = {p_value:.6f}")

# Step 3: Decision
if p_value < alpha:
    print("\nDecision: Reject H0 -> Yoga significantly reduced blood pressure.")
else:
    print("\nDecision: Fail to reject H0 -> No significant reduction in blood pressure.")

# Step 4: Graphs
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
for i in range(n):
    ax[0].plot(['Before', 'After'], [before[i], after[i]], marker='o', color='gray', alpha=0.7)
ax[0].plot(['Before', 'After'], [before.mean(), after.mean()], marker='s', color='red', lw=3,
label='Mean')
ax[0].set_title('BP of Each Patient Before and After Yoga')
ax[0].set_ylabel('Systolic BP (mmHg)'); ax[0].legend()

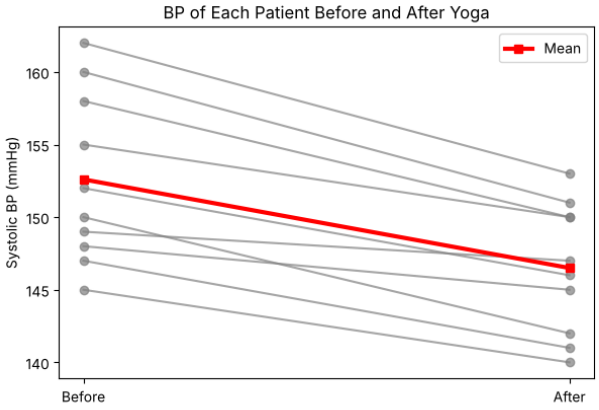
ax[1].bar(range(1, n+1), diff, color='seagreen', edgecolor='black')
ax[1].axhline(diff.mean(), color='red', linestyle='--', label=f'Mean reduction =
{diff.mean():.1f}')
ax[1].set_title('Reduction in BP for Each Patient')
ax[1].set_xlabel('Patient'); ax[1].set_ylabel('Before - After (mmHg)'); ax[1].legend()
plt.tight_layout()
plt.show()
```

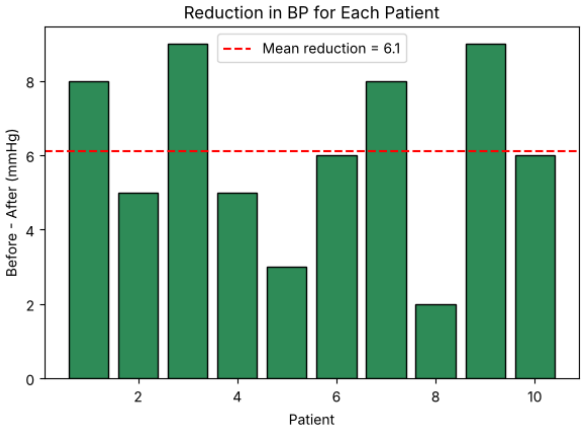
Output

```
Differences (Before - After): [8 5 9 5 3 6 8 2 9 6]
Mean difference = 6.10 mmHg, SD of differences = 2.42
t statistic = 7.9565
t critical  = 1.8331 (one-tailed, df = 9)
p-value     = 0.000012

Decision: Reject H0 -> Yoga significantly reduced blood pressure.
```

Graph





Result

The mean reduction in BP is 6.1 mmHg; $t = 7.96$ ($df = 9$), $p = 0.000012$. Since $t > 1.833$, H_0 is rejected. The yoga programme significantly reduced the systolic blood pressure of the patients, and every patient showed some reduction.

Program 8: F-Test for Equality of Two Variances

Aim: To test whether two populations have the same variance (consistency).

Scenario: A factory produces bolts of **10 mm** diameter on two machines. The quality manager feels **Machine A** is less consistent than **Machine B**. A sample of 10 bolts from each machine is measured. Do the two machines have equal variability? ($\alpha = 0.05$)

- H_0: $\sigma^2_A = \sigma^2_B$ H_1: $\sigma^2_A \neq \sigma^2_B$ (two-tailed)
- Test statistic: $\mathbf{F = s^2_1 / s^2_2}$ (larger variance in numerator), $df_1 = n_1 - 1$, $df_2 = n_2 - 1$
- *F-test is also the basis of ANOVA.*

Syntax

Syntax	Parameters	Why it is used in this program
np.var(a, ddof)	a – diameters from one machine ddof=1 – sample variance (divides by n − 1)	Finds the variance of each machine.
var_A / var_B	Larger variance is always put on top	Calculates the F statistic (F ≥ 1).
stats.f.cdf(x, dfn, dfd)	x – F statistic dfn – numerator df ($n_1 - 1 = 9$) dfd – denominator df ($n_2 - 1 = 9$)	Area to the left of F; p-value = 2 × (1 − cdf) for a two-tailed test.
stats.f.ppf(q, dfn, dfd)	q – $1 - \alpha/2 = 0.975$ dfn, dfd – degrees of freedom	Gives the F critical value 4.026.
stats.f.pdf(x, dfn, dfd)	x – array of F values	Gives the height of the F-curve so it can be drawn.

Program Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Step 1: Data - bolt diameters (mm)
machine_A = np.array([10.2, 9.8, 10.5, 9.6, 10.4, 9.9, 10.6, 9.7, 10.3, 10.1])
machine_B = np.array([10.0, 10.1, 9.9, 10.0, 10.2, 9.95, 10.05, 10.1, 9.9, 10.0])
alpha = 0.05

# Step 2: Sample variances
var_A = np.var(machine_A, ddof=1)
var_B = np.var(machine_B, ddof=1)

# Step 3: F statistic (larger variance / smaller variance)
if var_A >= var_B:
    F = var_A / var_B
    df1, df2 = len(machine_A) - 1, len(machine_B) - 1
```

```
else:
    F = var_B / var_A
    df1, df2 = len(machine_B) - 1, len(machine_A) - 1

p_value = 2 * (1 - stats.f.cdf(F, df1, df2))          # two-tailed
F_critical = stats.f.ppf(1 - alpha/2, df1, df2)

print(f"Variance of Machine A = {var_A:.4f}")
print(f"Variance of Machine B = {var_B:.4f}")
print(f"F statistic  = {F:.4f}  (df1 = {df1}, df2 = {df2})")
print(f"F critical   = {F_critical:.4f}")
print(f"p-value      = {p_value:.6f}")

# Step 4: Decision
if F > F_critical:
    print("\nDecision: Reject H0 -> The variances are different. Machine A is less
consistent.")
else:
    print("\nDecision: Fail to reject H0 -> Both machines have similar variability.")

# Step 5: Graphs
fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
ax[0].boxplot([machine_A, machine_B], patch_artist=True,
boxprops=dict(facecolor='khaki'))
ax[0].set_xticks([1, 2]); ax[0].set_xticklabels(['Machine A', 'Machine B'])
ax[0].axhline(10, color='red', linestyle='--', label='Target 10 mm')
ax[0].set_title('Spread of Bolt Diameters'); ax[0].set_ylabel('Diameter (mm)');
ax[0].legend()

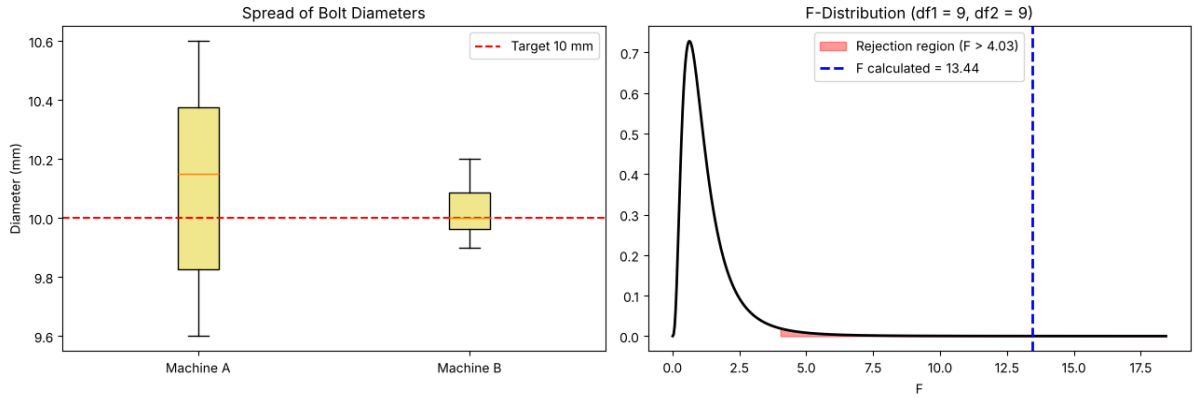
x = np.linspace(0.01, max(F + 5, 8), 500)
y = stats.f.pdf(x, df1, df2)
ax[1].plot(x, y, 'k', lw=2)
ax[1].fill_between(x, y, where=(x >= F_critical), color='red', alpha=0.4, label=f'Rejection
region (F > {F_critical:.2f})')
ax[1].axvline(F, color='blue', linestyle='--', lw=2, label=f'F calculated = {F:.2f}')
ax[1].set_title(f'F-Distribution (df1 = {df1}, df2 = {df2})')
ax[1].set_xlabel('F'); ax[1].legend()
plt.tight_layout()
plt.show()
```

Output

```
Variance of Machine A = 0.1210
Variance of Machine B = 0.0090
F statistic  = 13.4444  (df1 = 9, df2 = 9)
F critical   = 4.0260
p-value      = 0.000653

Decision: Reject H0 -> The variances are different. Machine A is less consistent.
```

Graph



Result

Variance of Machine A = 0.1210 and Machine B = 0.0090; F = 13.44, p = 0.00065. Since F > 4.026, H_0 is rejected. The variances are not equal: Machine A produces bolts with much more variation and is less consistent than Machine B.

Program 9: One-way ANOVA

Aim: To compare the means of more than two groups using one-way Analysis of Variance.

Scenario: An agricultural officer tests **three fertilizers (A, B, C)** on rice fields. Each fertilizer is used on **5 plots** and the yield (quintals per acre) is recorded. Do the fertilizers give different average yields? ($\alpha = 0.05$)

- H_0: $\mu_A = \mu_B = \mu_C$ H_1: at least one mean is different
- **F = MSB / MSW** = (variance between groups) / (variance within groups)
- df between = $k - 1$, df within = $N - k$
- *Why not three t-tests? Doing many t-tests increases the chance of a Type I error; ANOVA tests all groups at once.*

Syntax

Syntax	Parameters	Why it is used in this program
np.concatenate([a1, a2, a3])	a1, a2, a3 – the three fertilizer groups	Joins all 15 yields into one array to find the grand mean.
sum(len(g) * (g.mean() - grand_mean)**2 for g in groups)	len(g) – number of plots in the group g.mean() – group mean	Calculates SSB (variation between groups).
sum(((g - g.mean())**2).sum() for g in groups)	g – yields of one group	Calculates SSW (variation within groups).
stats.f.ppf(1 - alpha, dfn, dfd)	dfn – k − 1 = 2 dfd – N − k = 12	Gives the F critical value 3.885 (ANOVA is always right-tailed).
stats.f_oneway(a1, a2, a3)	a1, a2, a3 – the groups to compare Returns – F statistic and p-value	Checks the step-by-step ANOVA in one line.

Program Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Step 1: Data - yield (quintals/acre)
fert_A = np.array([20, 22, 19, 24, 21])
fert_B = np.array([25, 27, 24, 26, 28])
fert_C = np.array([22, 23, 21, 24, 22])
groups = [fert_A, fert_B, fert_C]
names = ['Fertilizer A', 'Fertilizer B', 'Fertilizer C']
alpha = 0.05

# Step 2: ANOVA table calculated step by step
```

```
all_data = np.concatenate(groups)
grand_mean = all_data.mean()
k, N = len(groups), len(all_data)

SSB = sum(len(g) * (g.mean() - grand_mean)**2 for g in groups)   # between groups
SSW = sum(((g - g.mean())**2).sum() for g in groups)              # within groups
SST = SSB + SSW
df_b, df_w = k - 1, N - k
MSB, MSW = SSB / df_b, SSW / df_w
F = MSB / MSW
p_value = 1 - stats.f.cdf(F, df_b, df_w)
F_critical = stats.f.ppf(1 - alpha, df_b, df_w)

print("ONE-WAY ANOVA TABLE")
print(f"{'Source':<16}{'SS':>10}{'df':>6}{'MS':>10}{'F':>10}")
print(f"{'Between groups':<16}{SSB:>10.2f}{df_b:>6}{MSB:>10.2f}{F:>10.3f}")
print(f"{'Within groups':<16}{SSW:>10.2f}{df_w:>6}{MSW:>10.2f}")
print(f"{'Total':<16}{SST:>10.2f}{N-1:>6}")
print(f"\nF critical = {F_critical:.3f},  p-value = {p_value:.6f}")

# Step 3: Check with scipy
F_scipy, p_scipy = stats.f_oneway(fert_A, fert_B, fert_C)
print(f"scipy f_oneway: F = {F_scipy:.3f}, p = {p_scipy:.6f}")

# Step 4: Decision
for name, g in zip(names, groups):
    print(f"  Mean yield of {name} = {g.mean():.2f}")
if p_value < alpha:
    print("\nDecision: Reject H0 -> At least one fertilizer gives a different average yield.")
else:
    print("\nDecision: Fail to reject H0 -> All fertilizers give the same average yield.")

# Step 5: Graph
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
ax[0].boxplot(groups, patch_artist=True, boxprops=dict(facecolor='lightgreen'))
ax[0].set_xticks([1, 2, 3]); ax[0].set_xticklabels(names)
ax[0].axhline(grand_mean, color='red', linestyle='--', label=f'Grand mean =
{grand_mean:.1f}')
ax[0].set_title('Crop Yield by Fertilizer'); ax[0].set_ylabel('Yield (quintals/acre)');
ax[0].legend()

means = [g.mean() for g in groups]
ax[1].bar(names, means, color=['#8fbc8f', '#2e8b57', '#9acd32'], edgecolor='black')
for i, m in enumerate(means):
    ax[1].text(i, m + 0.4, f'{m:.1f}', ha='center', fontweight='bold')
ax[1].set_title(f'Mean Yield (F = {F:.2f}, p = {p_value:.4f})'); ax[1].set_ylabel('Mean
yield')
plt.tight_layout()
plt.show()
```

Output

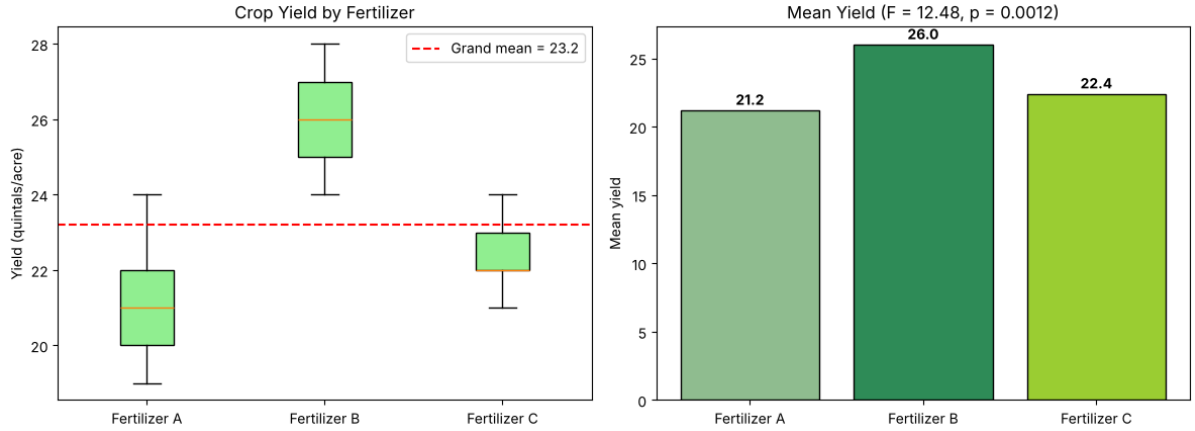
ONE-WAY ANOVA TABLE

Source	SS	df	MS	F
Between groups	62.40	2	31.20	12.480
Within groups	30.00	12	2.50	
Total	92.40	14		

F critical = 3.885, p-value = 0.001171
scipy f_oneway: F = 12.480, p = 0.001171
Mean yield of Fertilizer A = 21.20
Mean yield of Fertilizer B = 26.00
Mean yield of Fertilizer C = 22.40

Decision: Reject H0 -> At least one fertilizer gives a different average yield.

Graph



Result

F = 12.48 with df (2, 12), p = 0.0012. Since F > 3.885, H_0 is rejected. The fertilizers do not give the same average yield; Fertilizer B gives the highest mean yield (26.0 quintals/acre).

Program 10: Two-way ANOVA

Aim: To study the effect of two factors, and their interaction, on a response using two-way ANOVA.

Scenario: The agricultural officer now tests **3 fertilizers (A, B, C)** under **2 irrigation levels (Low, High)**. Each combination is tried on **3 plots** and the yield is recorded. ($\alpha = 0.05$)

Three sets of hypotheses are tested:

1. **Fertilizer effect** – H_0: all fertilizers give the same mean yield
2. **Irrigation effect** – H_0: both irrigation levels give the same mean yield
3. **Interaction effect** – H_0: the effect of fertilizer does not depend on irrigation level

Requires: `pip install statsmodels`

Syntax

Syntax	Parameters	Why it is used in this program
pd.DataFrame(data)	data – dictionary: column name → list of values	Stores Fertilizer, Irrigation and Yield as a table.
df.pivot_table(values, index, columns, aggfunc)	values – column to summarise (Yield) index – rows (Fertilizer) columns – columns (Irrigation) aggfunc='mean' – take the average	Shows the mean yield of each fertilizer–irrigation combination.
ols(formula, data).fit()	formula – 'Yield ~ C(Fertilizer) + C(Irrigation) + C(Fertilizer):C(Irrigation)' C( ) – treat the column as a categorical factor : – interaction between the two factors data – the DataFrame	Fits the two-way ANOVA model with interaction.
sm.stats.anova_lm(model, typ)	model – the fitted model typ=2 – Type II sums of squares	Produces the two-way ANOVA table (SS, df, F, p-value).
df.groupby(cols)[col].mean().unstack()	cols – ['Fertilizer', 'Irrigation'] col – 'Yield' unstack() – turns Irrigation into columns	Arranges the means for the interaction plot and bar chart.

Program Code

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.formula.api import ols

# Step 1: Data in table form
data = pd.DataFrame({
    'Fertilizer': ['A']*6 + ['B']*6 + ['C']*6,
    'Irrigation': (['Low']*3 + ['High']*3) * 3,
    'Yield': [20, 22, 21, 25, 27, 26,      # Fertilizer A
              24, 25, 23, 30, 31, 29,      # Fertilizer B
              22, 21, 23, 32, 33, 34]      # Fertilizer C
})
print(data.pivot_table(values='Yield', index='Fertilizer', columns='Irrigation',
aggfunc='mean').round(2))
alpha = 0.05

# Step 2: Two-way ANOVA with interaction
model = ols('Yield ~ C(Fertilizer) + C(Irrigation) + C(Fertilizer):C(Irrigation)',
data=data).fit()
anova_table = sm.stats.anova_lm(model, typ=2)
print("\nTWO-WAY ANOVA TABLE")
print(anova_table.round(4))

# Step 3: Decision for each factor
print()
for source in anova_table.index[:-1]:
    p = anova_table.loc[source, 'PR(>F)']
    decision = 'Significant (Reject H0)' if p < alpha else 'Not significant (Fail to reject H0)'
    print(f"{source:<30} p = {p:.6f} -> {decision}")

# Step 4: Graphs - interaction plot and grouped bar chart
means = data.groupby(['Fertilizer', 'Irrigation'])['Yield'].mean().unstack()[['Low', 'High']]
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
for fert in means.index:
    ax[0].plot(means.columns, means.loc[fert], marker='o', lw=2, label=f'Fertilizer {fert}')
ax[0].set_title('Interaction Plot (non-parallel lines = interaction)')
ax[0].set_xlabel('Irrigation level'); ax[0].set_ylabel('Mean yield'); ax[0].legend()

means.plot(kind='bar', ax=ax[1], color=['skyblue', 'navy'], edgecolor='black', rot=0)
ax[1].set_title('Mean Yield by Fertilizer and Irrigation')
ax[1].set_xlabel('Fertilizer'); ax[1].set_ylabel('Mean yield')
plt.tight_layout()
plt.show()
```

Output

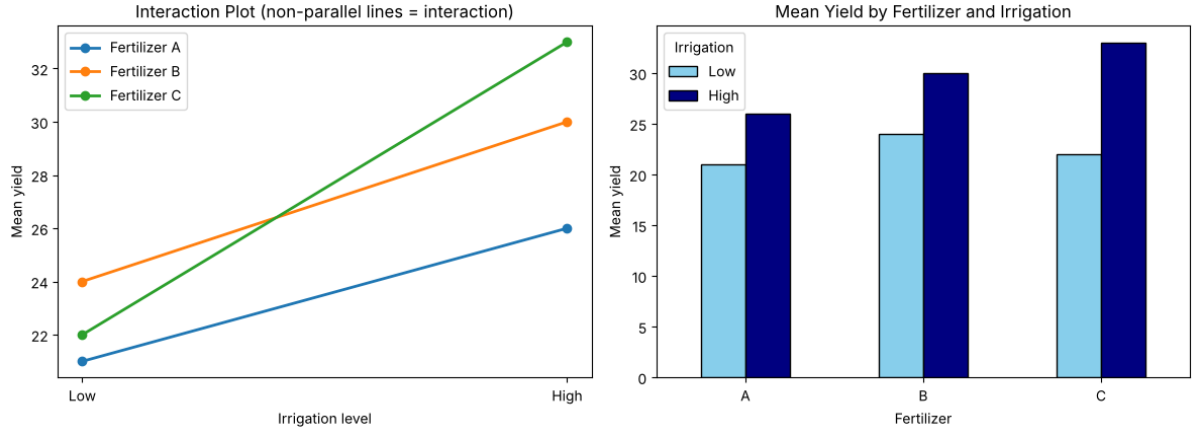
```
Irrigation  High   Low
Fertilizer
A           26.0  21.0
B           30.0  24.0
```

```
C          33.0  22.0

TWO-WAY ANOVA TABLE
                          sum_sq    df      F  PR(>F)
C(Fertilizer)               57.0   2.0   28.5  0.0000
C(Irrigation)              242.0   1.0  242.0  0.0000
C(Fertilizer):C(Irrigation)   31.0   2.0   15.5  0.0005
Residual                   12.0  12.0    NaN     NaN

C(Fertilizer)               p = 0.000028 -> Significant (Reject H0)
C(Irrigation)               p = 0.000000 -> Significant (Reject H0)
C(Fertilizer):C(Irrigation)    p = 0.000472 -> Significant (Reject H0)
```

Graph



Result

All three effects are significant ($p < 0.05$): Fertilizer ($F = 28.5$), Irrigation ($F = 242.0$) and the Fertilizer × Irrigation interaction ($F = 15.5$). High irrigation increases yield for all fertilizers, and the non-parallel lines in the interaction plot show that the benefit depends on the fertilizer: Fertilizer C gains the most (22 → 33). The best combination is Fertilizer C with high irrigation.

Program 11: Introduction to Chi-square Tests – Goodness of Fit

Aim: To test whether observed frequencies match expected frequencies.

Scenario: In a board-game club, players complain that a die is **biased**. The die is rolled **120 times**. If it is fair, each face should appear 120/6 = **20 times**. Is the die fair? ($\alpha = 0.05$)

About Chi-square (χ^2) tests

- Used for **categorical (count) data**, not means.
- $\chi^2 = \Sigma (O - E)^2 / E$ where O = observed, E = expected frequency.
- Goodness of fit: df = k − 1. Test of independence (Program 12): df = (r − 1)(c − 1).
- Condition: every expected frequency should be at least 5.
- H_0: The die is fair (all faces equally likely) H_1: The die is not fair

Syntax

Syntax	Parameters	Why it is used in this program
stats.chisquare(f_obs, f_exp)	f_obs – observed frequencies [25, 17, 15, 23, 24, 16] f_exp – expected frequencies (20 each) Returns – χ^2 statistic and p-value	Performs the chi-square goodness of fit test.
stats.chi2.ppf(q, df)	q – 1 − α = 0.95 df – k − 1 = 5	Gives the χ^2 critical value 11.07.
stats.chi2.pdf(x, df)	x – array of χ^2 values	Gives the height of the χ^2 curve so it can be drawn.
ax.bar(x, height, width, label)	x – positions (shifted −0.2 and +0.2) width – bar width (0.4)	Places observed and expected bars side by side for each face.

Program Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Step 1: Data
faces = ['1', '2', '3', '4', '5', '6']
observed = np.array([25, 17, 15, 23, 24, 16])
expected = np.array([sum(observed) / 6] * 6)
alpha = 0.05

# Step 2: Calculation table
print(f"{'Face':>5}{'O':>6}{'E':>8}{'(O-E)^2/E':>12}")
for f, o, e in zip(faces, observed, expected):
    print(f"{f:>5}{o:>6}{e:>8.1f}{(o-e)**2/e:>12.3f}")
```

```
# Step 3: Chi-square test
chi2_stat, p_value = stats.chisquare(f_obs=observed, f_exp=expected)
df = len(observed) - 1
chi2_critical = stats.chi2.ppf(1 - alpha, df)
print(f"\nChi-square statistic = {chi2_stat:.4f}")
print(f"Degrees of freedom   = {df}")
print(f"Chi-square critical  = {chi2_critical:.4f}")
print(f"p-value              = {p_value:.4f}")

# Step 4: Decision
if p_value < alpha:
    print("\nDecision: Reject H0 -> The die is biased.")
else:
    print("\nDecision: Fail to reject H0 -> The die is fair; differences are due to chance.")

# Step 5: Graphs
fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
x = np.arange(6)
ax[0].bar(x - 0.2, observed, width=0.4, label='Observed', color='coral', edgecolor='black')
ax[0].bar(x + 0.2, expected, width=0.4, label='Expected', color='lightgray',
edgecolor='black')
ax[0].set_xticks(x); ax[0].set_xticklabels(faces)
ax[0].set_title('Observed vs Expected Frequencies'); ax[0].set_xlabel('Die face');
ax[0].set_ylabel('Frequency')
ax[0].legend()

xc = np.linspace(0, 20, 500)
yc = stats.chi2.pdf(xc, df)
ax[1].plot(xc, yc, 'k', lw=2)
ax[1].fill_between(xc, yc, where=(xc >= chi2_critical), color='red', alpha=0.4,
label=f'Rejection region (> {chi2_critical:.2f})')
ax[1].axvline(chi2_stat, color='blue', linestyle='--', lw=2, label=f'χ² calculated =
{chi2_stat:.2f}')
ax[1].set_title(f'Chi-square Distribution (df = {df})'); ax[1].set_xlabel('χ²'); ax[1].legend()
plt.tight_layout()
plt.show()
```

Output

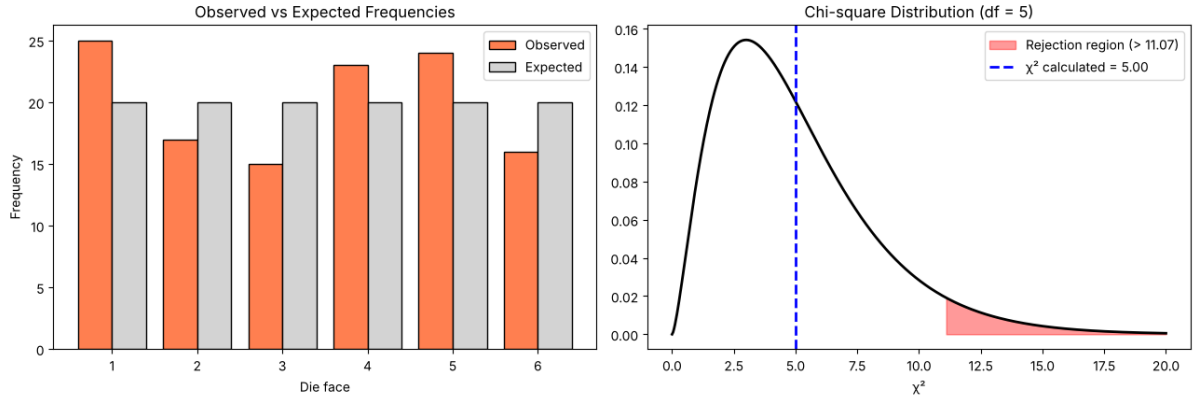
```
 Face     O       E   (O-E)^2/E
    1    25    20.0       1.250
    2    17    20.0       0.450
    3    15    20.0       1.250
    4    23    20.0       0.450
    5    24    20.0       0.800
    6    16    20.0       0.800

Chi-square statistic = 5.0000
Degrees of freedom   = 5
Chi-square critical  = 11.0705
```

p-value = 0.4159

Decision: Fail to reject H0 -> The die is fair; differences are due to chance.

Graph



Result

$\chi^2 = 5.00$ (df = 5), p = 0.416. Since $\chi^2 < 11.07$ (p > 0.05), H_0 is not rejected. The die is fair; the differences between observed and expected counts are due to chance.

Program 12: Chi-square Test of Independence

Aim: To test whether two categorical variables are associated.

Scenario: A college surveys **200 students** to find out whether **preferred learning mode** (Online, Offline, Hybrid) depends on **gender**. ($\alpha = 0.05$)

	Online	**Offline**	**Hybrid**	**Total**
Male	30	45	25	100
Female	50	25	25	100

- H_0: Gender and learning-mode preference are **independent** (not related)
- H_1: Gender and learning-mode preference are **associated**
- Expected frequency E = (Row total × Column total) / Grand total

Syntax

Syntax	**Parameters**	**Why it is used in this program**
pd.DataFrame(data, index, columns)	data – list of rows of counts index – row names (Male, Female) columns – column names (Online, Offline, Hybrid)	Creates the contingency table.
stats.chi2_contingency(observed)	observed – the contingency table Returns – χ^2 statistic, p-value, df and expected table	Performs the chi-square test of independence and finds the expected frequencies.
stats.chi2.ppf(q, df)	q – 0.95 df – $(r-1)(c-1) = 2$	Gives the χ^2 critical value 5.991.
table.T.plot(kind, ax)	T – swaps rows and columns kind='bar' – bar chart ax – which graph to draw on	Draws the grouped bar chart of preferences by gender.
ax.imshow(X, cmap)	X – table of counts cmap='Blues' – colour scale	Draws a heatmap of the counts.
ax.text(x, y, s, ha, va, color)	x, y – position s – text to write ha, va – horizontal and vertical alignment	Writes the observed (O) and expected (E) counts in each cell.

Program Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from scipy import stats

# Step 1: Contingency table
table = pd.DataFrame([[30, 45, 25],
                      [50, 25, 25]],
                     index=['Male', 'Female'],
                     columns=['Online', 'Offline', 'Hybrid'])
print("Observed frequencies:")
print(table)
alpha = 0.05

# Step 2: Chi-square test of independence
chi2_stat, p_value, df, expected = stats.chi2_contingency(table)
print("\nExpected frequencies:")
print(pd.DataFrame(expected, index=table.index, columns=table.columns).round(2))

chi2_critical = stats.chi2.ppf(1 - alpha, df)
print(f"\nChi-square statistic = {chi2_stat:.4f}")
print(f"Degrees of freedom   = {df}")
print(f"Chi-square critical  = {chi2_critical:.4f}")
print(f"p-value              = {p_value:.6f}")

# Step 3: Decision
if p_value < alpha:
    print("\nDecision: Reject H0 -> Learning-mode preference depends on gender.")
else:
    print("\nDecision: Fail to reject H0 -> Learning-mode preference is independent of
gender.")

# Step 4: Graphs
fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
table.T.plot(kind='bar', ax=ax[0], color=['steelblue', 'palevioletred'], edgecolor='black',
rot=0)
ax[0].set_title('Preferred Learning Mode by Gender')
ax[0].set_xlabel('Learning mode'); ax[0].set_ylabel('Number of students')

im = ax[1].imshow(table.values, cmap='Blues')
ax[1].set_xticks(range(3)); ax[1].set_xticklabels(table.columns)
ax[1].set_yticks(range(2)); ax[1].set_yticklabels(table.index)
for i in range(2):
    for j in range(3):
        color = 'white' if table.values[i, j] > 40 else 'black'
        ax[1].text(j, i, f'O={table.values[i, j]}\nE={expected[i, j]:.0f}', ha='center',
va='center', fontsize=11, color=color)
ax[1].set_title('Observed (O) vs Expected (E) Counts')
plt.colorbar(im, ax=ax[1])
plt.tight_layout()
plt.show()
```

Output

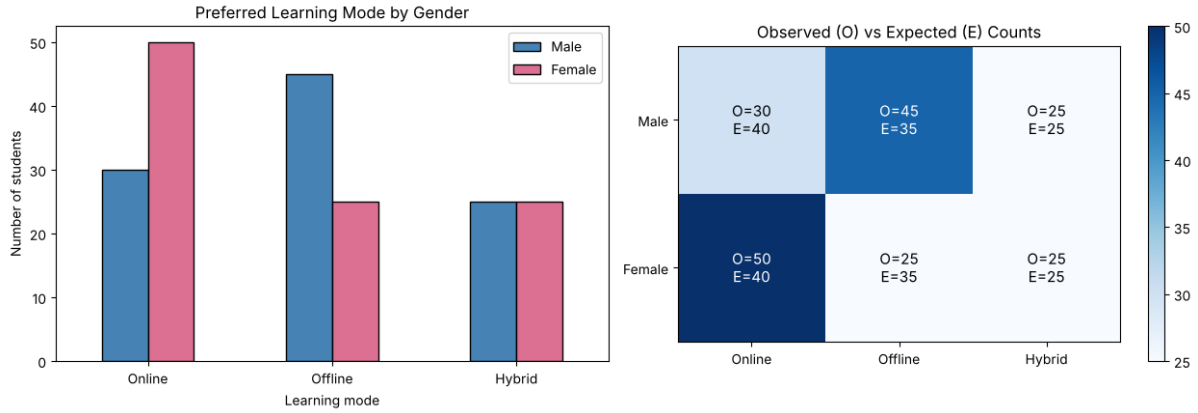
```
Observed frequencies:
        Online  Offline  Hybrid
Male        30       45      25
Female      50       25      25

Expected frequencies:
        Online  Offline  Hybrid
Male      40.0     35.0    25.0
Female    40.0     35.0    25.0

Chi-square statistic = 10.7143
Degrees of freedom   = 2
Chi-square critical  = 5.9915
p-value              = 0.004714

Decision: Reject H0 -> Learning-mode preference depends on gender.
```

Graph



Result

$\chi^2 = 10.71$ (df = 2), p = 0.0047. Since $\chi^2 > 5.991$, H_0 is rejected. Preferred learning mode is associated with gender: more female students prefer online learning, while more male students prefer offline learning; hybrid preference is the same for both.